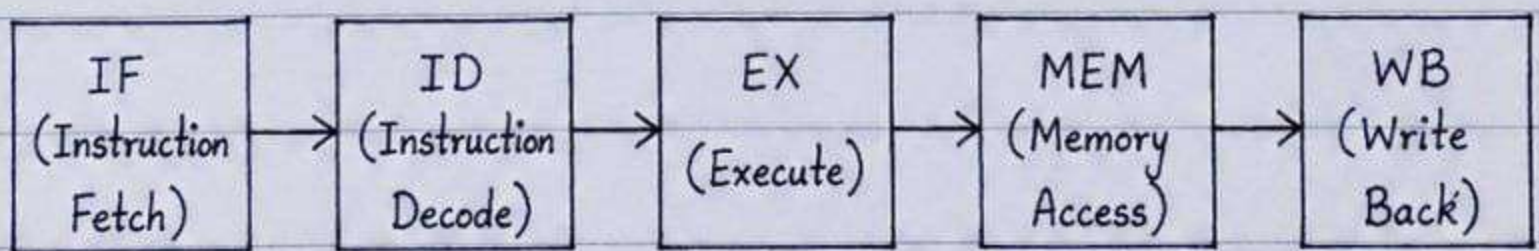


Does pipelining reduce the execution time of a single machine instruction? Explain. (3 marks)

Ans;

- No, pipelining does not reduce the execution time of a single machine instruction.
- The execution time of one instruction is equal to the slowest stage delay of the pipeline.
- Pipelining improves performance only by increasing the throughput (more instructions completed per unit time), not the latency of a single instruction.



Time for a single instruction
= Delay of the slowest stage

Compare and contrast pipelined and single-cycle instruction execution. Use timing diagrams and MIPS examples; mention stage delays and speedup. (5 marks)

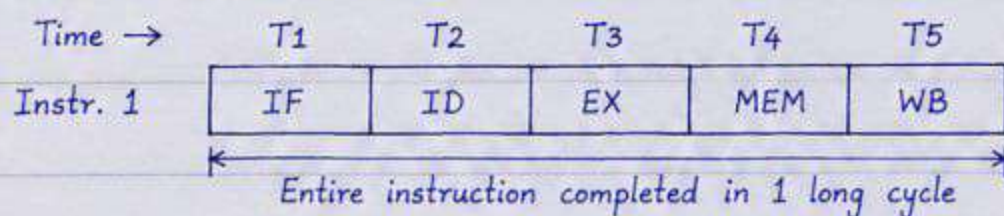
Ans;

1. Comparison:

Single-Cycle Execution	Pipelined Execution
- Entire instruction completes in one long clock cycle. - Clock period = sum of all stage delays + overhead. - Low hardware complexity. - Lower throughput.	- Instruction is divided into stages; each stage takes one clock cycle. - Clock period = delay of the slowest stage + overhead. - Higher hardware complexity. - Higher throughput (after pipeline fill).

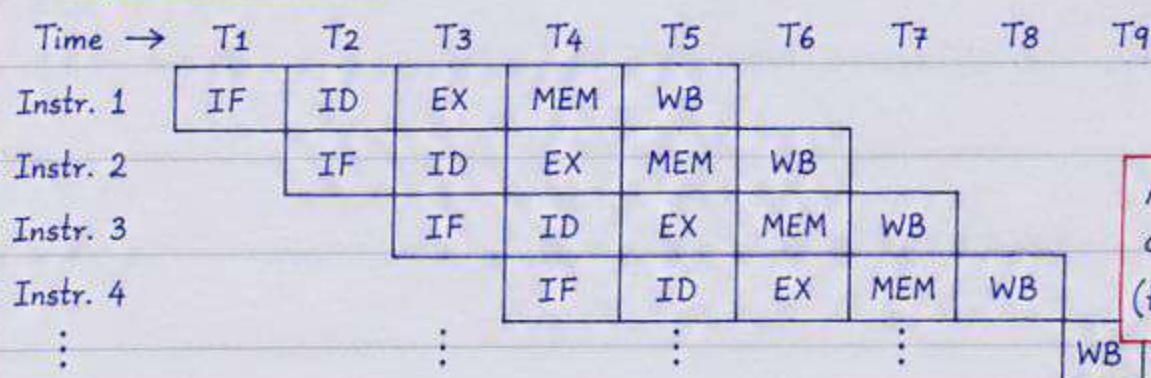
2. Timing Diagrams (5-stage MIPS Pipeline):

(a) Single-Cycle Execution



$$\text{Cycle time} = T_{IF} + T_{ID} + T_{EX} + T_{MEM} + T_{WB} + T_{ovh}$$

(b) Pipelined Execution



After fill, one instruction completes every cycle (throughput = 1 instr./cycle)

3. Stage Delays (MIPS 5-stage example):

Stage	IF	ID	EX	MEM	WB	Overhead
Delay (ns)	200	150	200	250	150	50

- Slowest stage = 250 ns (MEM)
- Pipelined clock period = 250 ns + 50 ns (ovh) = 300 ns
- Single-cycle clock period = (200 + 150 + 200 + 250 + 150) + 50 = 1000 ns

4. Speedup:

- For n instructions and k stages,
- Single-cycle time = $n \times (\sum \text{stage delays} + \text{overhead})$
- Pipelined time = $(k + n - 1) \times (T_{\text{slowest}} + \text{overhead})$

$$\text{Speedup} = \frac{T_{\text{single-cycle}}}{T_{\text{pipelined}}}$$

What factors limit the ideal speedup of a pipeline? (3-5 marks)

Ans:

The ideal speedup of a pipeline (= number of stages) is limited in practice by the following factors:

- ① **Unequal stage delays**: If the stages are unbalanced, the clock cycle is determined by the **slowest stage**, so the speedup is less than the number of stages.
- ② **Pipeline overhead**: Extra time needed for pipeline registers, latches and control increases the **clock cycle time**, reducing speedup.
- ③ **Pipeline filling and draining**: During start-up and completion, some stages remain idle. For n instructions and k stages, only $(n + k - 1)$ cycles are used, not n cycles.
- ④ **Hazards**: Data, control and structural hazards cause **stalls** or **bubbles** in the pipeline, decreasing the effective throughput and speedup.
- ⑤ **Resource limitations**: Memory bandwidth, functional units and I/O constraints may not support the required throughput, limiting speedup.

* Note (Ideal vs Actual Speedup):

Ideal speedup = number of stages (k)

Actual speedup = $\frac{k}{1 + (k-1) \times \frac{\text{stall cycles}}{n}}$ [n = number of instructions]

		5-stage pipeline ($k=5$)							
Instruction	Cycle $\rightarrow$	1	2	3	4	5	6	7	8
I_1		IF	ID	EX	MEM	WB			
I_2			IF	ID	EX	MEM	WB		
I_3				IF	ID	EX	MEM	WB	
I_4					IF	ID	EX	MEM	WB
I_5						IF	ID	EX	WB

← Total cycles = $n + k - 1 = 5 + 5 - 1 = 9$ cycles →

Explain structural, data, and control hazards with suitable examples. (5 marks)

Ans:

In pipelined processors, hazards are conditions that prevent the next instruction in the pipeline from executing in its designated clock cycle.

1. Structural Hazard: It occurs when two or more instructions compete for the same hardware resource in the same clock cycle.

Example: A single memory is used for both instruction fetch and data access. If a load instruction needs data memory access in MEM stage while the next instruction needs instruction fetch in the same cycle, a structural hazard occurs.

Solution: Use separate instruction and data memories or duplicate required hardware.

2. Data Hazard: It occurs when an instruction depends on the result of a previous instruction that is still in the pipeline.

Example:
 I_1 : ADD R1, R2, R3 ; $R_1 = R_2 + R_3$
 I_2 : SUB R4, R1, R5 ; uses R1

I_2 needs the value of R1 produced by I_1 , which is not yet available in the register.

Solution: Use forwarding (bypassing) or insert stalls (NOPs).

3. Control Hazard: It occurs when the next instruction to be executed is unknown due to a branch or jump instruction.

Example:
 I_1 : BEQ R1, R2, Label ; Branch instruction
 I_2, I_3 : Next sequential instructions

The pipeline does not know the branch outcome or target address until the branch is resolved.

Solution: Use branch prediction, delayed branch, or flush incorrect instructions.

Summary:

Hazard	Cause	Example Situation	Remedies
Structural	Competition for the same hardware resource	Memory used for both instruction fetch and data access	Separate resources or duplicate hardware
Data	Dependency on result of previous instruction	Instruction uses register value not yet written	Forwarding or stalls
Control	Uncertainty in next PC due to branch/jump	Branch instruction in pipeline	Branch prediction or flush

How do structural hazards occur? How does MIPS design avoid them? (3-5 marks)

Ans:

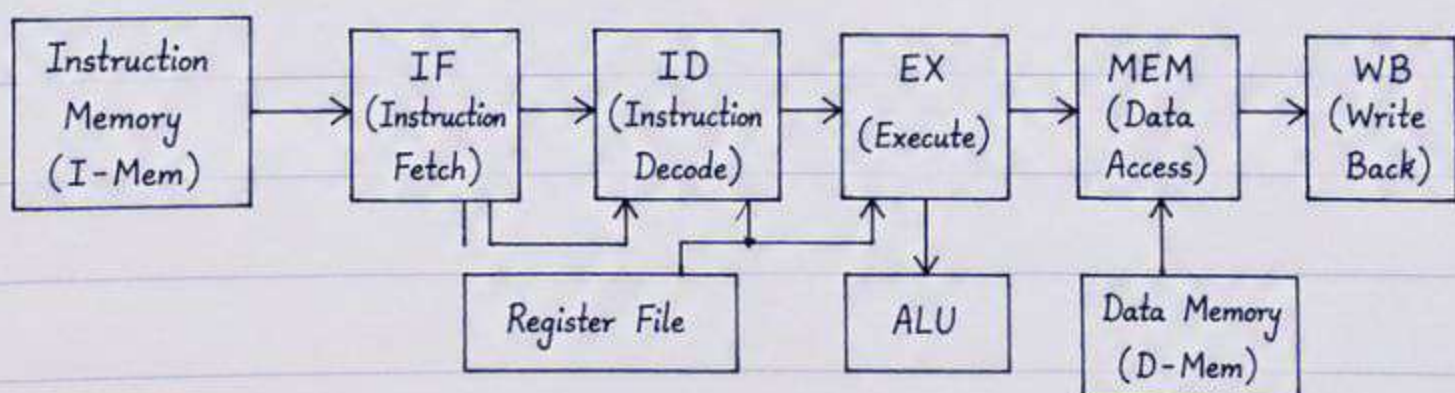
Structural hazards occur when two or more instructions compete for the same hardware resource in the same clock cycle.

Example: If both an instruction fetch and a data access try to use the same memory in the same cycle, a structural hazard occurs.

MIPS Design Avoidance: MIPS avoids structural hazards by using separate hardware resources so that multiple operations can occur in the same cycle.

- Separate Instruction Memory (I-Mem) for instruction fetch.
- Separate Data Memory (D-Mem) for load/store access.
- Separate register file with enough ports for simultaneous read and write.
- Separate ALU for execution.
- Dedicated adders for $PC+4$ and branch target calculation.

Thus, no two instructions compete for the same resource, and structural hazards are avoided in MIPS pipeline.



What is a control hazard? Why does it mostly happen with branch instructions? (3-5 marks)

Ans:

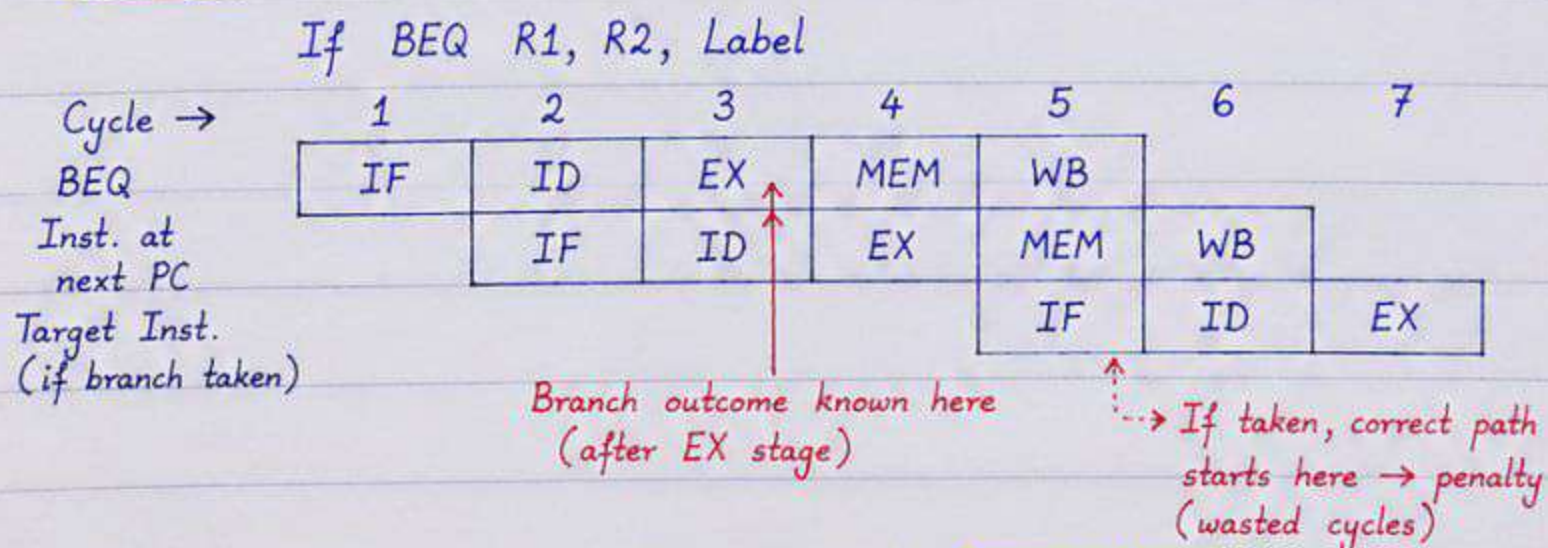
1. Control Hazard:

A control hazard occurs when the next instruction to be executed is not known at the time of instruction fetch due to a branch or jump instruction.

2. Why does it mostly happen with branch instructions?

- For branch instructions, the decision (taken or not taken) depends on a condition or comparison result.
- This result is available only after the instruction reaches the EX stage.
- Until then, the processor does not know the correct next PC (address).
- Therefore, instructions fetched after a branch may be wrong (from the wrong path), causing control hazard.

3. Example:



Thus, control hazards mainly occur with branch/jump instructions due to the unknown next PC until the branch decision is resolved.

Analyze how control signals are generated and propagated through the pipeline datapath; explain how incorrect control handling leads to hazards. (5 marks)

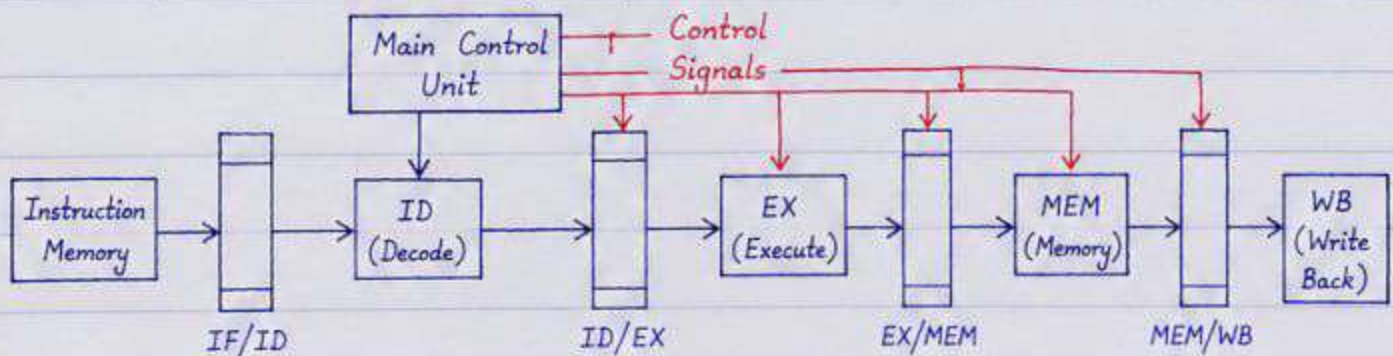
Ans:

1. Generation of control signals:

- The opcode of the instruction is read in the IF stage.
- It is sent to the main control unit (in ID stage).
- The control unit decodes the opcode and generates control signals such as RegWrite, ALUSrc, MemRead, MemWrite, Branch, Jump, ALUOp etc.
- These signals determine the operations in EX, MEM and WB stages.

2. Propagation through the pipeline:

- Control signals must travel with the instruction through pipeline registers (ID/EX, EX/MEM, MEM/WB).
- Each pipeline register holds both data and the corresponding control signals.
- In every stage, the control signals are used to enable/disable hardware components (ALU, memory, register file, mux, etc.).



3. How incorrect control handling leads to hazards:

- If wrong control signals are applied to a stage, the processor performs an incorrect operation.
- This causes control hazards, e.g., when a branch is taken but instructions after the branch have already entered the pipeline.

• Effects:

Wrong data written, wrong memory access, incorrect register update, or execution of instructions that should have been discarded.

• Reason:

Delay in branch decision, missing flush, or not blocking control signals for wrong-path instructions.

• Remedy:

Use proper control logic → Stall, Flush, Forward/Override control signals to maintain correct flow.

Explain pipeline stalls caused by data hazards. Propose two techniques to resolve or minimize them. (5 marks)

Ans:

Data hazards occur when an instruction depends on the result of a previous instruction that is still in the pipeline. Until the required data is available, the dependent instruction cannot proceed, causing a **pipeline stall** (bubble).

Example: I_1 : ADD R1, R2, R3 ; I_2 : SUB R4, R1, R5

Here, I_2 needs R1 produced by I_1 . If R1 is not ready, I_2 must wait $\rightarrow$ **pipeline stall**.

Pipeline stall due to data hazard (example):

Cycle $\rightarrow$	1	2	3	4	5	6	7
I_1 (ADD)	IF	ID	EX	MEM	WB		
I_2 (SUB)		IF	ID	STALL	STALL	EX	MEM

I_2 is **stalled** in ID until R1 becomes available (after EX/MEM).

Techniques to resolve or minimize stalls:

1. Forwarding (Bypassing):

- The result is **forwarded** directly from a later stage (EX/MEM or MEM/WB) to the ALU input of a **dependent instruction**.
- **Eliminates stalls** for most data hazards (except load-use hazard).
- Uses multiplexers and forwarding paths controlled by **forwarding unit**.

2. Stall Insertion (Pipeline Interlock):

- Hardware detects the hazard and inserts one or more **bubbles** by freezing PC and IF/ID register and converting control signals to **NOP**.
- **Simple and reliable**; used when forwarding cannot resolve the hazard (e.g., load-use case).

Summary: Data hazards force the pipeline to wait (stall) when needed data is not ready. **Forwarding** minimizes stalls by supplying data early, while **stall insertion (interlock)** ensures **correct execution** when forwarding is not possible.

For:

```
lw $t1, 0($t2)
add $t3, $t1, $t4
sub $t5, $t3, $t6
```

→ Identify hazard type, show stall/forwarding, check if reordering helps. (5 marks)

Ans:

1. Identify hazard type:

- $lw\ \$t1, 0(\$t2) \rightarrow add\ \$t3, \$t1, \$t4$:
Load-Use data hazard (RAW).
- $add\ \$t3, \$t1, \$t4 \rightarrow sub\ \$t5, \$t3, \$t6$:
ALU-ALU data hazard (RAW) on $\$t3$.

2. Stall / Forwarding (Without reordering):

(a) With one stall (for load-use) + forwarding

Cycle →	1	2	3	4	5	6	7	8
$lw\ \$t1, 0(\$t2)$	IF	ID	EX	MEM	WB			
$add\ \$t3, \$t1, \$t4$		IF	ID (stall)	ID	EX	MEM	WB	
$sub\ \$t5, \$t3, \$t6$				IF	ID	EX	MEM	WB

- One stall inserted after ID of add (cycle 3).
- Forwarding used:
 - $\$t1$ forwarded from MEM/WB of lw (cycle 4) to EX of add (cycle 5).
 - $\$t3$ forwarded from EX/MEM of add (cycle 5) to EX of sub (cycle 6).

3. Reordering to reduce/avoid stall:

Reorder by moving an independent instruction between lw and add.

Example:

```
lw $t1, 0($t2)
add $t7, $t8, $t9 # independent
add $t3, $t1, $t4
sub $t5, $t3, $t6
```

Now load-use hazard disappears.
Forwarding handles the rest.
No stall required.

Cycle →	1	2	3	4	5	6	7	
lw \$t1, 0(\$t2)	IF	ID	EX	MEM	WB			
add \$t7, \$t8, \$t9		IF	ID	EX	MEM	WB		
add \$t3, \$t1, \$t4			IF	ID	EX	MEM	WB	
sub \$t5, \$t3, \$t6				IF	ID	EX	MEM	WB

- $\$t1$ forwarded from MEM/WB of lw to EX of add.
- $\$t3$ forwarded from EX/MEM of add to EX of sub.

Conclusion: Load-use hazard causes one stall; forwarding eliminates remaining data hazards. Reordering with an independent instruction avoids the stall.

Why is a stall still required even when forwarding is used?

Explain with a load instruction example. (3-5 marks)

Ans:

- **Forwarding** can pass results from later stages (EX/MEM/WB) to earlier stages to avoid stalls.
- But in a **load instruction**, the **data is available** only after the MEM stage.
- The next instruction needs that data in its **EX stage** in the very next cycle.
- Since the data from load is **not ready** until the end of MEM, it cannot be forwarded in time for the next instruction's EX.
- Therefore, **one stall (bubble)** is required even with forwarding.

Example: lw \$t1, 0(\$t2) # load word
 add \$t3, \$t1, \$t4 # uses loaded data

Pipeline Timing (with forwarding but with stall):

Cycle →	1	2	3	4	5	6
lw \$t1, 0(\$t2)	IF	ID	EX	MEM	WB	
add \$t3, \$t1, \$t4		IF	ID (stall)	ID (stall)	EX	MEM

↑
Data available after MEM,
forwarded to EX in cycle 5

Explanation:

- Load data from memory is ready at the **end of MEM** (cycle 4).
- The add needs the data at the **beginning of its EX** (cycle 4).
- Hence, add is **stalled** in ID for one cycle.
- In cycle 5, the data is **forwarded** from MEM/WB of load to EX of add.

Conclusion: Forwarding cannot eliminate the **load-use hazard** completely; **one stall** is still required.

How does instruction reordering help reduce stalls and performance loss? (5 marks)

Ans:

- Instruction reordering means changing the order of independent instructions (without changing program meaning).
- It helps reduce stalls caused by data hazards by scheduling independent instructions between dependent ones.
- This allows the pipeline to stay busy and improves performance.
- It is done by the compiler (static reordering) or by hardware (dynamic scheduling).

Example: Original code (causes stall)

1. lw \$t1, 0(\$t2) # load word
2. add \$t3, \$t1, \$t4 # uses loaded data
3. sub \$t5, \$t3, \$t6 # uses result of add
4. and \$t7, \$t8, \$t9 # independent instruction

Pipeline timing (original):

Cycle →	1	2	3	4	5	6	7	8
lw \$t1, 0(\$t2)	IF	ID	EX	MEM	WB			
add \$t3, \$t1, \$t4		IF	ID (stall)	ID (stall)	EX	MEM	WB	
sub \$t5, \$t3, \$t6				IF	ID	EX	MEM	WB

↑ Two stall cycles

Reordered code (moves independent instruction up):

1. lw \$t1, 0(\$t2) # load word
2. and \$t7, \$t8, \$t9 # independent
3. add \$t3, \$t1, \$t4 # uses loaded data
4. sub \$t5, \$t3, \$t6 # uses result of add

Pipeline timing (reordered):

Cycle →	1	2	3	4	5	6	7
lw \$t1, 0(\$t2)	IF	ID	EX	MEM	WB		
and \$t7, \$t8, \$t9		ID	ID	EX	MEM	WB	
add \$t3, \$t1, \$t4			IF	ID	EX	MEM	WB
sub \$t5, \$t3, \$t6				IF	ID	EX	MEM

- No stall cycles in this schedule.
- Pipeline remains full → better performance (less CPI).

Conclusion: Instruction reordering reduces or eliminates stalls by filling pipeline bubbles with useful work, thus reducing performance loss.

How can you resolve data hazards? Explain with diagram. (5 marks)

Ans:

Data hazards occur when an instruction depends on the result of a previous instruction still in the pipeline.

Methods to resolve data hazards:

1. Forwarding (Bypassing):

- The result is forwarded directly from a later stage (EX/MEM/WB) to an earlier stage (usually EX) of the dependent instruction.
- Eliminates most RAW hazards without stalling.

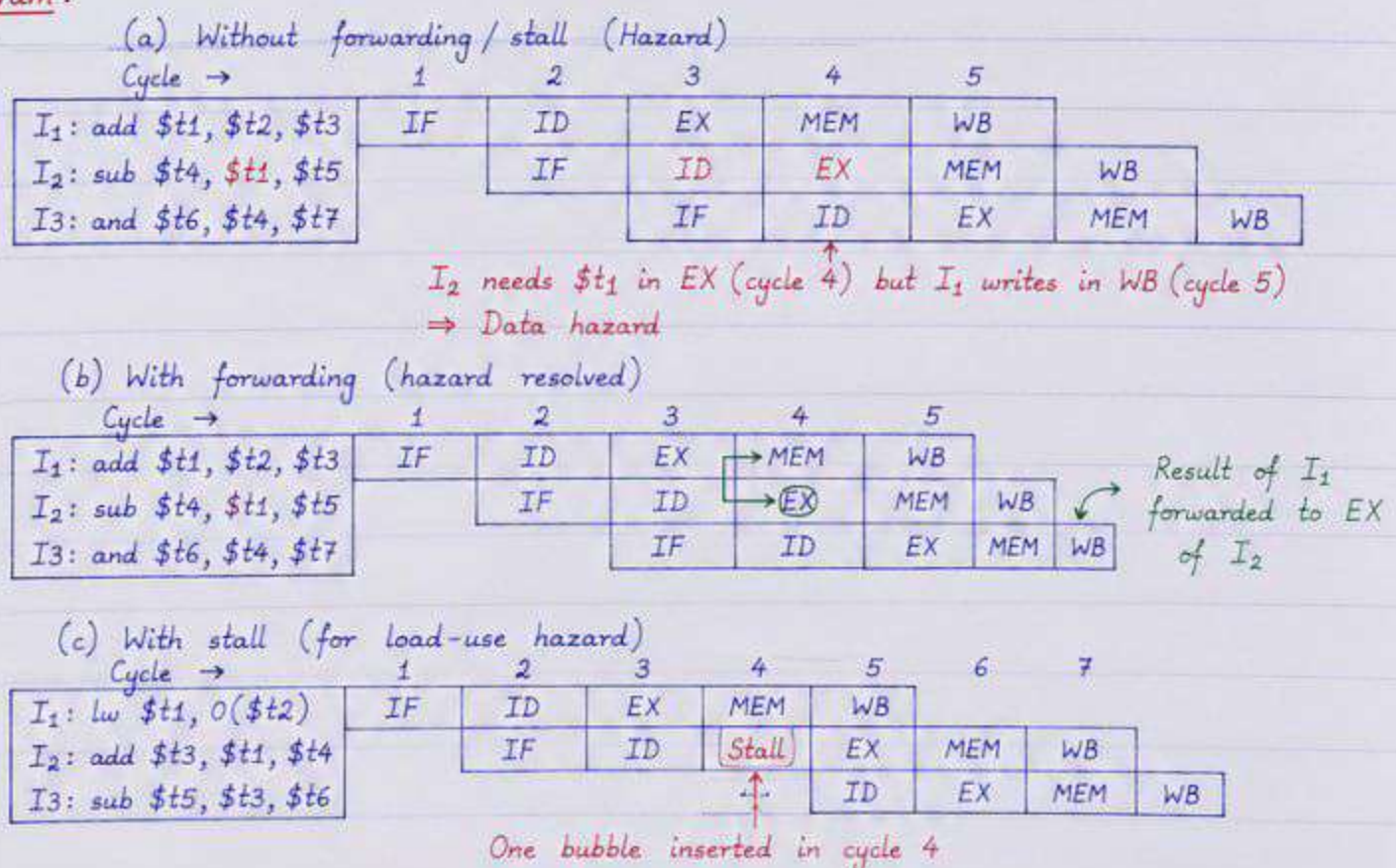
2. Stall / Pipeline Interlock):

- Hardware detects a hazard that cannot be solved by forwarding (e.g., load-use hazard).
- Inserts bubbles (NOPs) into the pipeline to delay the dependent instruction until the data is available.

3. Instruction Reordering (Compiler scheduling):

- Compiler rearranges independent instructions between dependent ones to reduce or avoid hazards.
- Reduces the need for stalls and improves performance.

Diagram:



Conclusion: Data hazards can be resolved using forwarding, stalling (interlocks), and instruction reordering. Forwarding removes most hazards; stalling handles the rest; reordering reduces stalls.

Analyze how stalls are introduced due to dependent instructions and how reordering minimizes performance loss. (5 marks)

Ans:

In a pipeline, when an instruction depends on the result of a previous instruction that is not yet available, the pipeline must wait. This waiting is called a **stall** (or bubble). Reordering independent instructions between dependent ones reduces or eliminates these stalls and thus improves performance.

1. Stalls due to dependent instructions (RAW hazard):

- A dependent instruction needs a result that is produced by an earlier instruction.
- Since the data is not ready in time, following instructions are delayed → **stall** is inserted.

Example:

I_1 : add \$t1, \$t2, \$t3 # produces \$t1
 I_2 : sub \$t4, \$t1, \$t5 # uses \$t1 (dependent)
 I_3 : and \$t6, \$t7, \$t8 # independent

Pipeline timing with stall (assuming 5-stage pipeline):

Cycle →	1	2	3	4	5	6	7	
I_1 : add \$t1, \$t2, \$t3	IF	ID	EX	MEM	WB			
I_2 : sub \$t4, \$t1, \$t5		IF	Stall	Stall	EX	MEM	WB	
I_3 : and \$t6, \$t7, \$t8				IF	ID	EX	MEM	WB

Two stall cycles are inserted so that I_2 gets \$t1 after I_1 completes EX.

2. How reordering minimizes performance loss:

- By moving independent instructions between dependent ones, we can utilize the pipeline and **hide** the latency.
- This reduces or eliminates the need for stalls.

Reordered code:

I_1 : add \$t1, \$t2, \$t3 # produces \$t1
 I_3 : and \$t6, \$t7, \$t8 # independent (moved up)
 I_2 : sub \$t4, \$t1, \$t5 # uses \$t1

Pipeline timing after reordering:

Cycle →	1	2	3	4	5	6	
I_1 : add \$t1, \$t2, \$t3	IF	ID	EX	MEM	WB		
I_3 : and \$t6, \$t7, \$t8		IF	ID	EX	MEM	WB	
I_2 : sub \$t4, \$t1, \$t5			IF	ID	EX	MEM	WB

No stall cycles. The dependent instruction I_2 gets the data in EX using **forwarding** from I_1 .

Conclusion: Stalls are introduced when dependent instructions need data not yet available. Instruction reordering places independent instructions in between to keep the pipeline busy and **minimize performance loss**.

Q. Draw and explain the 5-stage pipelined MIPS datapath.

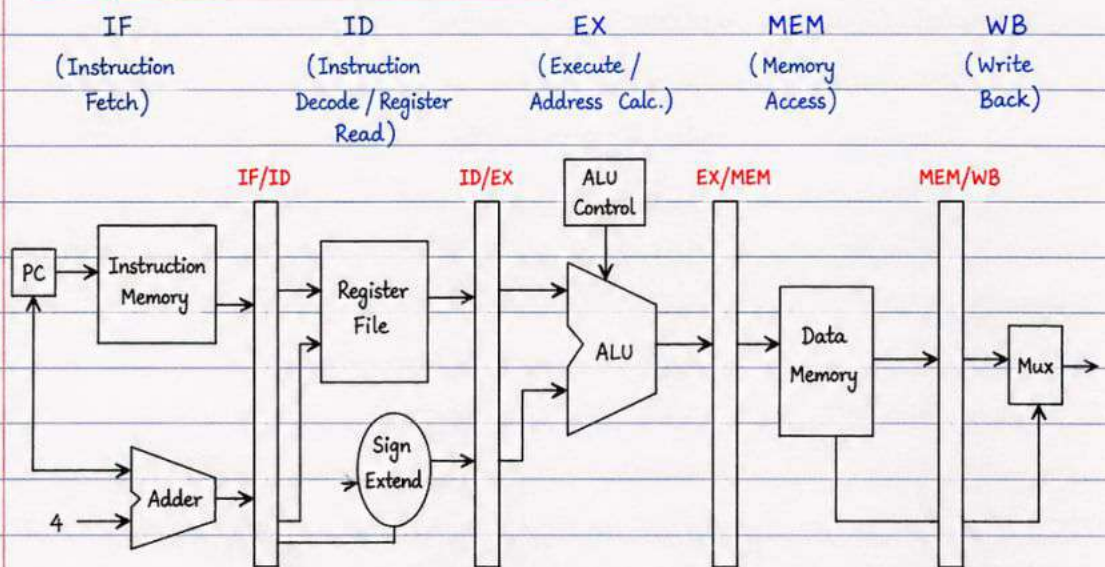
What are pipeline registers and why are they needed? (5 marks)

Answer :

5-stage pipelined MIPS datapath :

- Instruction execution is divided into 5 stages.
- Multiple instructions are overlapped in different stages.
- This increases throughput (≈ 1 instruction per clock) after pipeline is filled.

5-stage pipelined MIPS datapath :



Pipeline registers and their need :

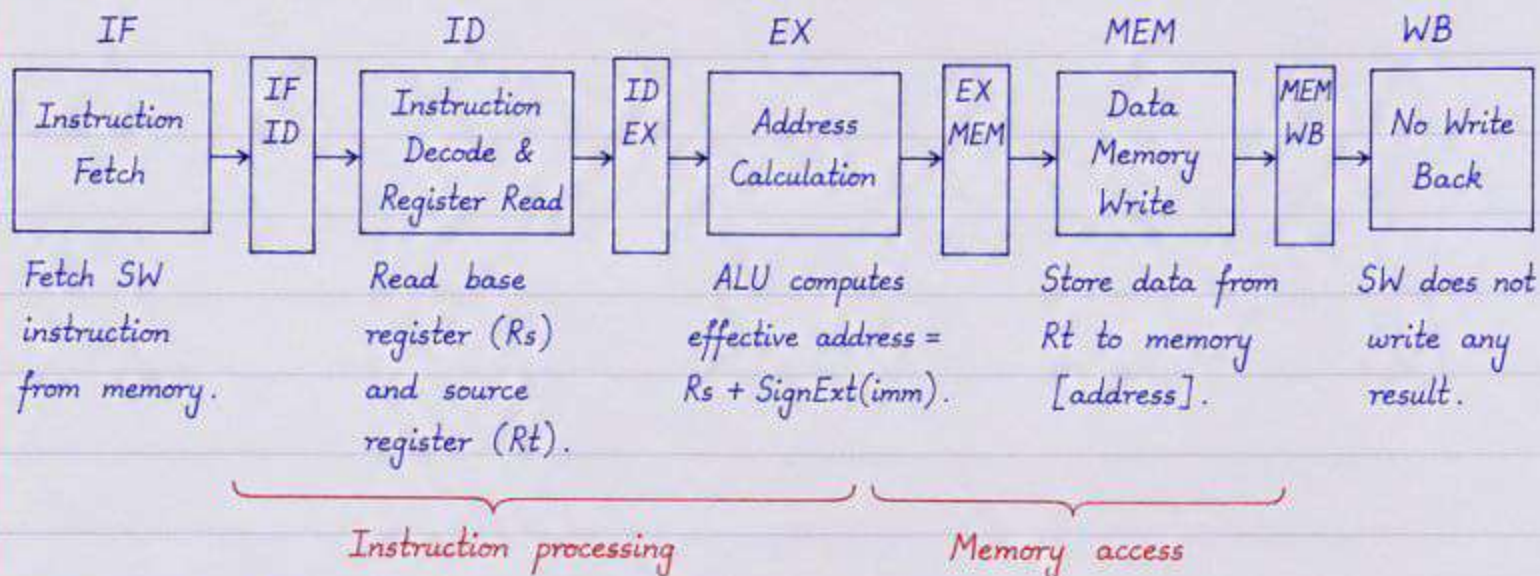
- Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) are special purpose registers placed between two stages.
- They store intermediate results (data, control signals, PC, etc.) at the end of a clock cycle.
- Need : They isolate stages, synchronize data transfer and allow different instructions to be in different stages simultaneously without interfering.
- Thus, they enable pipelining and improve performance.

Draw and explain the pipeline stages involved in executing a SW (Store Word) instruction; discuss how hazards are handled during execution. (5 marks)

Ans:

SW (Store Word) stores the content of a register into memory.

1. Pipeline stages for SW instruction:



Stage explanation (brief):

- **IF**: Fetch the SW instruction using PC.
- **ID**: Decode instruction, read base register R_s and source register R_t .
- **EX**: ALU adds base register and sign-extended immediate to get effective address.
- **MEM**: Write the data from R_t to memory at the computed address.
- **WB**: No write-back, since SW does not produce a result.

2. Hazards in SW instruction:

- **Data hazards**:
 - If R_s (base) or R_t (data) is produced by a previous instruction.
- **Structural hazards**:
 - Occur if memory or register file is busy.
- **Control hazards**:
 - Only if a branch/jump appears before SW, causing flush.

3. How hazards are handled:

Hazard Type	Handling Technique	Notes
Data hazard (R_s or R_t)	Forwarding	Forward base address or store data from EX/MEM or MEM/WB to EX stage.
Load-use (R_t from load)	Stall (1 bubble)	If R_t is loaded in previous instruction, data is available only after MEM stage.
Structural hazard	Stall	Insert bubble if memory is busy.
Control hazard	Flush / Stall	If branch is taken, flush wrong-path SW.

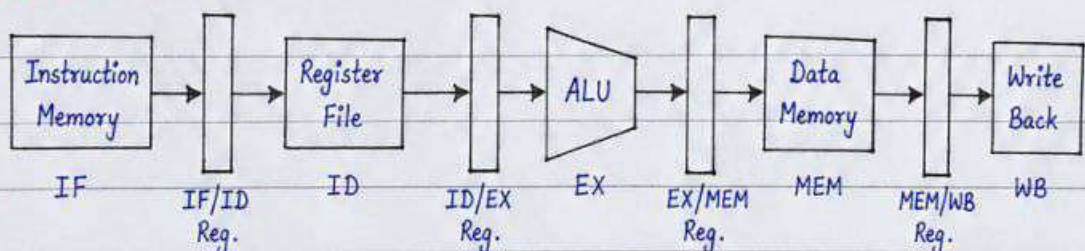
Conclusion: SW uses all five pipeline stages. Hazards are mainly data hazards on R_s or R_t , handled by forwarding or stalls to ensure correct memory write.

Q. What changes are required when moving from single-cycle to pipelined datapath? (3-5 marks)

Ans: To convert a single-cycle datapath into a pipelined datapath, the following changes are required:

- Partition the datapath into stages (IF, ID, EX, MEM, WB) so that each stage performs a part of the instruction execution.
- Insert pipeline registers between stages (e.g., IF/ID, ID/EX, EX/MEM, MEM/WB) to hold intermediate data and control signals.
- Modify the control unit to generate stage-wise control signals and pass them through the pipeline registers.
- Handle data hazards by adding forwarding/bypassing hardware or by inserting stalls.
- Handle control hazards using techniques like stalling or flushing (e.g., with branch prediction).

Pipeline view:



→ These changes allow multiple instructions to be executed concurrently in different stages, increasing throughput.

Q. Why is dynamic pipelining more complicated than traditional/static pipelining? (2 marks)

Answer :

- Static pipelining has fixed stage order and deterministic timing, so hardware is simple with minimal control.
- Dynamic pipelining allows variable stage latency and out-of-order completion, requiring complex hardware for monitoring, buffering, hazard detection, scheduling and result reordering.

Q. Briefly explain superscalar and superpipelining techniques. (3 marks)

1. Superscalar Technique:

- Superscalar processor can issue and execute multiple instructions simultaneously in one clock cycle.
- It uses multiple functional units (e.g., ALU, FPU, Load/Store units) to execute instructions in parallel.
- Increases instruction throughput and overall CPU performance.

2. Superpipelining Technique:

- Superpipelining divides each pipeline stage into smaller sub-stages to increase the number of pipeline stages.
- It reduces the critical path delay, allowing a shorter clock cycle time.
- Improves instruction throughput but increases hardware complexity and pipeline overhead.